
CSC 4200 / 5200 — Computer Networks

Lab 0: Course Workstation Setup

Due: End of Week 2 · Estimated time: 30–45 minutes
Tennessee Tech University · Department of Computer Science · Fall 2026

You will need: A reliable Internet connection and about 20 GB of free disk space.

1. Why You Are Doing This

Later in the semester you will write a TCP server that accepts multiple clients at once, a client that talks to it, and a custom protocol header that you pack and parse byte by byte. You will also open packet captures and work out what went wrong inside them.

None of that is exotic, but all of it depends on having a working C compiler, the OpenSSL development headers, `make`, `git`, and a handful of network utilities that are all present and all agreeing with each other. Getting that combination right on 60 different laptops is where courses like this lose two weeks.

So instead we all build the same machine. A virtual machine is a computer running inside your computer. It has its own disk, its own network interface, and its own copy of Linux. When it breaks — and something will break — you delete it and rebuild it in twenty minutes without touching anything you care about.

Do this now, in Week 1 or 2

Not the night before Programming Assignment 1 is due. The whole point of doing it early is that if it fails, there is time to fix it.

2. Phase 1: Download What You Need

A. The Operating System

Download the **Ubuntu Server 24.04 LTS** ISO image. It is roughly 2 GB.

- Link: <https://ubuntu.com/download/server>
- Apple Silicon Macs: download the **ARM64** image. Everyone else: **AMD64**.

We use Ubuntu Server rather than Ubuntu Desktop. Server has no graphical interface, which makes it smaller and faster, and you will be editing your code from VS Code on your own machine anyway (Phase 5).

B. The Hypervisor

The hypervisor is the program that runs the virtual machine.

Path A — VirtualBox (everyone: Windows, Intel Mac, Apple Silicon Mac)

Use **VirtualBox 7.2 or later**. It is free, and since version 7.2 it runs ARM guests natively on Apple Silicon, so Windows PCs and M-series Macs now follow the same instructions.

- Download: <https://www.virtualbox.org/wiki/Downloads>
- **Apple Silicon Macs (M1 and later):** download the package whose filename contains `macOSArm64`. Do not grab the Intel build.
- **Windows note:** if the VM refuses to start, you probably need to enable virtualization (VT-x or AMD-V) in your BIOS or UEFI settings.

The one rule that trips people up on Apple Silicon

An ARM host runs ARM guests. Your Ubuntu ISO must be the **ARM64** image. If you download the AMD64 image on an M-series Mac, the VM will not boot and the error message will not tell you why. Check this before you spend an hour on it.

VirtualBox on Apple Silicon still cannot run x86-64 guests, and its 3D acceleration is experimental. Neither matters here — we are running a headless Linux server with no graphics.

Path B — UTM (alternative for Apple Silicon Macs)

If VirtualBox gives you trouble on your Mac, **UTM** is a well-supported alternative that has been running ARM64 Linux on Apple Silicon since long before VirtualBox could.

- Download: <https://mac.getutm.app/>

Either hypervisor is acceptable. Phases 3 through 5 are identical once Ubuntu is installed.

Path C — Windows with WSL2 (alternative, no VM required)

If you already run WSL2, or you are short on disk space, WSL2 will do everything this course requires. Install Ubuntu 24.04 from the Microsoft Store or run `wsl --install -d Ubuntu-24.04`, then skip to **Phase 4**.

Two things to know before you choose this path:

- Your code still has to build and run for the grader on a clean Ubuntu 24.04 machine. WSL2 is close enough that this is rarely a problem, but you own the difference.
- WSL2 has no `systemd` by default, so the SSH service will not start. That is fine — you do not need SSH at all on this path. See Phase 5 for how to connect VS Code.

If WSL2 gives you trouble, fall back to Path A. Course support is written against the VM.

C. Wireshark, on your own computer

Later in the semester you will analyze packet capture files. The capture files are supplied by the instructor, so you do not need capture permissions — you just need something to open them in.

- Download: <https://www.wireshark.org/download.html> — install this on your own operating system, not inside the VM.

Your VM gets `tshark`, the command-line version, for scripted work. The point-and-click version lives on your machine where it will actually be pleasant to use.

3. Phase 2: Create the Virtual Machine

Settings (both paths)

Setting	Value
CPU	2 cores
RAM	4096 MB (2048 MB works if your laptop is tight on memory)
Disk	20 GB, dynamically allocated
Network	NAT is fine. Bridged is slightly more convenient for SSH.

Path A: VirtualBox

1. Open VirtualBox, click **New**.
2. **Name:** `CSC4200-Workstation`
3. **Type:** Linux. **Subtype:** Ubuntu. **Version:** Ubuntu (64-bit) on Intel/AMD, or **Ubuntu (ARM 64-bit)** on Apple Silicon.
4. **ISO Image:** the Ubuntu 24.04 ISO you downloaded — matching your architecture.
5. Check **Skip Unattended Installation** — you want to see the installer.
6. **Hardware:** 4 GB RAM, 2 CPUs.
7. **Hard Disk:** create a 20 GB virtual disk.
8. **Finish**, then **Start**.

Path B: UTM (Apple Silicon alternative)

1. Open UTM, choose **Create a New Virtual Machine**.
2. Select **Virtualize** (not Emulate). Emulate is dramatically slower and you do not need it.
3. Select **Linux**.
4. **Boot Image:** browse to your Ubuntu Server ARM64 ISO.
5. **Hardware:** 4 GB RAM, 2 cores.

6. **Storage:** 20 GB.
7. **Save**, then run.

4. Phase 3: Install Ubuntu

1. **Boot:** follow the prompts. English → Continue without updating → Done.
2. **Network:** it should pick up an address automatically via DHCP.
3. **Storage:** choose "Use an entire disk." This is the *virtual* 20 GB disk, not your real hard drive.
4. **Profile setup:** your actual name; server name `net-box`; username `student` (or your own preference — just remember it); a password you will remember, written down somewhere.
5. **SSH setup — do not skip this.** When asked "Install OpenSSH server?", check **[X] Install OpenSSH server**. VS Code needs it in Phase 5.
6. **Featured server snaps:** select none.
7. **Reboot.** In VirtualBox you may have to remove the ISO from the Devices → Optical Drives menu if it loops back into the installer.

You will end up at a black screen asking for your username and password. That is correct. That is what a server looks like.

5. Phase 4: Run the Setup Script

Log in to your VM. Now you will install the course toolchain.

Step 1 — Clone the course repository

The course repository is public, so you do not need a GitHub account or a login to get it.

```
cd ~
git clone https://github.com/elbrown-tnitech/csc4200.git csc4200-course
cd csc4200-course/lab0
```

That last word on the clone line, `csc4200-course`, is deliberate. It names the folder the repository lands in and keeps course materials separate from your own work. Section 8 explains why that separation matters.

Linux cares about capital letters. Your laptop does not.

macOS and Windows treat `Csc4200`, `CSC4200`, and `csc4200` as the same name. **Linux treats them as three different names.** A path that works on your laptop can fail inside the VM for no

reason you can see, and the error — `No such file or directory` — does not tell you that capitalization was the problem.

Everything in this course is **lowercase**: the repository, the folders, the commands. If you type a path and Linux says it does not exist, check your capital letters before you check anything else. Tab completion is your friend here — type a few letters and press `Tab`, and the shell fills in the rest with the capitalization it actually finds on disk.

Step 2 — Read the script before you run it

```
less setup_vm.sh
```

Press `q` to exit when you are done. This takes two minutes and it is not busywork: you are about to give a script your root password, and in a networking and security course you should never be in the habit of executing something you have not looked at. Skim what it installs and what it changes.

Step 3 — Run it

```
chmod +x setup_vm.sh
./setup_vm.sh
```

Enter your password when prompted. The script takes 10–15 minutes, mostly downloading packages. It writes a full transcript to `~/csc4200-setup.log`.

If it fails, it will stop and tell you where. Re-running it is safe — it skips work already completed. If it fails twice, post `~/csc4200-setup.log` in the class Teams channel and we will sort it out.

Step 4 — Verify

```
cd ~/csc4200-work
source venv/bin/activate
python3 ~/csc4200-course/lab0/verify_env.py
```

The `source venv/bin/activate` line matters. Without it, Python cannot see the libraries the script installed and the check will fail for no good reason.

The verification script does more than confirm files exist. It compiles and runs a small C program that uses `pthread`s, `OpenSSL`, and `sockets` together, builds a target from a `Makefile`, and opens a TCP connection to itself. Those three tests are the ones that actually predict whether Programming Assignment 1 will work.

When it finishes, it writes `~/csc4200-work/lab0_report.txt`.

6. Submitting Lab 0

What to submit

Upload `lab0_report.txt` to the Lab 0 assignment in Canvas.

Full marks require every check to report `[OK]`. If something reports `[FAIL]`, fix it and re-run — but if you are stuck, submit the report as-is before the deadline and bring it to office hours. A failing report submitted on time is far more useful to me than nothing, because it tells me exactly what broke.

To get the report out of the VM, run `cat ~/csc4200-work/lab0_report.txt` and copy the text from your terminal window. If you have already connected VS Code (Phase 5), you can open the file in the editor and copy it from there instead.

7. Phase 5: Connect VS Code

You will not write code inside the VM's black-and-white console window. You will use VS Code on your own machine to edit files that live inside the VM.

That setup has its own handout, because it is where most people get stuck and the failures are hard to tell apart from one another:

→ Connecting VS Code to Your Lab VM (Remote-SSH)

On Canvas alongside this lab. It covers host prerequisites for Windows and macOS, the hypervisor networking options and which one to pick, SSH key setup, opening files in the editor from the VM's terminal, and a troubleshooting section organized by the error message you actually see.

Two things from it are worth knowing before you start, because they cause most first-attempt failures:

- **The address you connect to is not the one `ip addr` shows.** On the default NAT setup, the host reaches the VM at `127.0.0.1` on a forwarded port, not at the `10.0.2.15` you see inside the VM.
- **A 1 GB VM will fail even though it boots fine.** The VS Code server needs about 1 GB of available RAM and 500 MB of disk. If you followed the specs in Phase 2 you already have enough.

On the WSL2 path? Skip Remote-SSH entirely. Install Microsoft's **WSL** extension and connect with `F1` → **WSL: Connect to WSL**. There is no VM and no SSH involved, so most of the supplement does not apply to you.

8. Where Your Work Lives

The setup script left you with two folders in your home directory. Keeping them straight is the single most useful habit you can build this week, because every assignment for the rest of the term assumes it.

Folder	What it is
~/csc4200-course/	Course materials — labs, starter code, capture files. A copy of the class repository. You never edit anything in here.
~/csc4200-work/	Everything you write. Your workspace. This is where your assignment repositories go.

One sentence version: **materials come out of the course folder, your work goes in the work folder, and the two never mix.**

Starting a work session

Two commands:

```
cd ~/csc4200-work
source venv/bin/activate
```

The first moves you to your workspace. The second puts the course's Python libraries on your path — your prompt gains a `(venv)` prefix so you can see it worked.

If you are writing C or C++, skip the second command. The virtual environment only affects Python. Nothing in the C toolchain needs it.

Getting updated course materials

When new labs or starter files are posted, pull them:

```
cd ~/csc4200-course
git pull
```

This overwrites the course folder with the current version. That is why you do not edit files there — your changes would be lost, and `git pull` would refuse to run until you undid them. When an assignment gives you starter code, **copy it into your workspace first:**

```
cp -r ~/csc4200-course/pal ~/csc4200-work/pal-starter
```

Assignment repositories

Programming Assignment 1 and Programming Assignment 2 each require a private GitHub repository of your own, with a real commit history. The assignment brief will give you the exact name to use — for example `csc4200-pa1-jsmith`, using your own GitHub username, all lowercase.

Clone your assignment repository **inside** your workspace, so all your work sits in one place:

```
cd ~/csc4200-work
git clone https://github.com/yourusername/csc4200-pa1-yourusername.git
```

Never run `git init` in `~/csc4200-work`

Your workspace is an ordinary folder, not a repository, and it must stay that way. Each assignment repository is its own separate clone inside it.

If you turn the workspace itself into a repository, your assignment repositories end up nested inside another one. Git handles that badly, your commit history stops recording what you think it records, and **both programming assignments award 20 of 100 points for commit history**. If you are ever unsure, run `git status` in `~/csc4200-work` — it should say it is not a git repository. That is the correct answer.

What it all looks like

<code>~/csc4200-course/</code>	course materials (git pull only)
<code>~/csc4200-work/</code>	
<code>venv/</code>	Python environment
<code>captures/</code>	capture files for analysis work
<code>scratch/</code>	experiments, throwaway code
<code>lab0_report.txt</code>	your Lab 0 submission
<code>csc4200-pa1-jsmith/</code>	your PA1 repository
<code>csc4200-pa2-jsmith/</code>	your PA2 repository

Tools you now have, and when you will want them

Tool	Use it for
<code>gcc, make</code>	Building your assignments
<code>gdb, valgrind</code>	Finding the segfault, finding the leak
<code>nc (netcat)</code>	Talking to your server <i>before</i> your client works
<code>tcpdump, tshark</code>	Watching your own traffic on the loopback interface
<code>ping, traceroute, dig, mtr</code>	Diagnosing reachability, path, and name resolution
<code>ss</code>	Confirming your server is actually listening on the port you think
<code>git</code>	Version control — graded on both programming assignments

`nc` is the one students underuse. If your server is not responding, `nc localhost 8080` tells you in five seconds whether the problem is in the server or in the client. That single habit will save you hours.

9. Scope and Acceptable Use

These rules are narrow and not negotiable

This VM contains real network tools. The following apply to every assignment, lab, and project in this course.

- **Capture traffic on your own loopback interface only**, or open capture files supplied by the instructor.
- **Do not capture on the campus network**, on residential networks you share with other people, or on any network you do not own.
- **Do not scan, probe, or intercept traffic** directed at hosts or services that are not yours.
- **Use synthetic data** in anything your programs log or transmit. No real passwords, no personal information.

The encryption examples in this course are teaching examples. Several use a hard-coded key or a fixed initialization vector so the mechanics are visible. That is not a pattern to carry into anything real, and the assignments will say so where it matters.

If you are ever unsure whether something is in scope, ask before you run it. Asking is always the right answer.

10. Troubleshooting

"The VM will not start" (Windows). Virtualization is disabled in your BIOS or UEFI. Reboot into firmware settings and enable VT-x (Intel) or AMD-V (AMD).

"The VM will not boot on my M-series Mac." Nine times out of ten this is the wrong ISO. Confirm you downloaded the **ARM64** Ubuntu image and the **macOSArm64** VirtualBox package. An ARM host cannot boot an AMD64 guest.

"VirtualBox is unusably slow on my Mac." Confirm you are on VirtualBox 7.2 or later — earlier versions emulated x86 on Apple Silicon instead of virtualizing ARM, which is very slow. If it is still sluggish after that, switch to UTM (Path B) and make sure you chose **Virtualize** rather than Emulate.

"Every Python library says MISSING." You did not activate the virtual environment. Run `source ~/csc4200-work/venv/bin/activate` and try again.

"The C toolchain check fails." Re-run `./setup_vm.sh`; the OpenSSL development headers probably did not install. The last few lines of `~/csc4200-setup.log` will say why.

"tshark says I don't have permission." Log out and back in. Group membership only takes effect on a new login session.

"VS Code cannot connect." Work through the troubleshooting section of the *Connecting VS Code to Your Lab VM* supplement — it sorts the fixes by the exact error message you are seeing. Before you do anything else, confirm SSH is running inside the VM with `systemctl status ssh`, since nothing else will work until it is.

"I broke it completely." Delete the VM and start over from Phase 2. This takes about thirty minutes and is a perfectly respectable option. Being able to throw the machine away is the reason we use one.

You are now ready for the programming assignments.